```
free(p);                                    /* freeing memory from
heap */

printf("freeing memory...\n");

return 0;

}
```

When we compile and run the above program, the output is as shown below:

```
[root@workbenchsvr malloc_hook]# gcc -o prog1 prog1.c
[root@workbenchsvr malloc_hook]# ./prog1
calling from main...
returning to main...
freeing memory...
[root@workbenchsvr malloc_hook]#
```

The next program, called *prog2.c*, is a simple hook for the *malloc()* function:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdint.h>
#include <dlfcn.h>                                          /*
header required for dlsym() */

/* lcheck() is for memory leak check; its code is not shown
 here */

void lcheck(void);

void* malloc(size_t size)

{

    static void* (*my_malloc)(size_t) = NULL;

    printf("inside shared object...\n");

    if (!my_malloc)

        my_malloc = dlsym(RTLD_NEXT, "malloc");  /* returns the
object reference for malloc */

    void *p = my_malloc(size);                          /* call
malloc() using function pointer my_malloc */

    printf("malloc(%d) = %p\n", size, p);

    lcheck();
/* calling do_your_stuff function */

    printf("returning from shared object...\n");

    return p;

}

void lcheck(void)

{

printf("displaying memory leaks...\n");

     /* do required stuff here */

}
```

Compiling and running the above, goes like this:

```
[root@workbenchsvr malloc_hook]# gcc -shared -ldl -fPIC prog2.c
-o libprog2.so

[root@workbenchsvr malloc_hook]# LD_PRELOAD=/home/dibyendu/
malloc_hook/libprog2.so ./prog1

calling from main...

inside shared object...

malloc(10) = 0x8191008

displaying memory leaks...
```

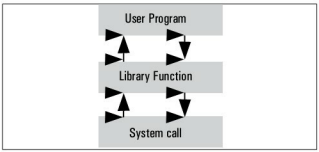

Figure 1: Library function

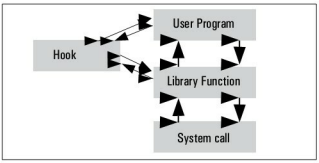

Figure 2: Library function with hook